

RAG Assignment

Section A: Vector Databases

A1. Explain in your own words what a vector database is and how it's different from a traditional database like MySQL or PostgreSQL.

A vector database is a database used to store and search **embeddings**. Embeddings are numerical representations of data such as text, images, or audio. A vector database is mainly useful when we want to find information based on its meaning or similarity.

Traditional databases such as MySQL and PostgreSQL usually store structured data such as names, numbers, dates, and other records. They are generally used for exact searches.

For example, a traditional database can be used to find a student whose roll number is 43. A vector database, on the other hand, can find documents that are semantically similar to a question such as "How can I reset my password?" even if the exact words are not present in the document.

A2. Take any sentence of your choice. Describe what happens to it step by step from raw text to being stored in a vector database. Name each stage.

Consider the sentence:

"I Love Cats and wish to be a cat."

The sentence goes through the following stages:

1. **Raw Text:** We start with the original sentence.
2. **Parsing:** Unnecessary spaces, formatting, or other unwanted content can be cleaned.
3. **Chunking:** If the text is part of a large document, it is divided into smaller chunks.
4. **Embedding:** An embedding model converts the text or chunk into a numerical vector.
5. **Vector Storage:** The generated vector is stored in the vector database along with the original text and useful metadata.
6. **Indexing:** The vector database creates an index so that similar vectors can be found efficiently during a search.

The overall process is:

Raw Text $\rightarrow$ Parsing $\rightarrow$ Chunking $\rightarrow$ Embedding $\rightarrow$ Vector Storage $\rightarrow$ Indexing

A3. Why can't we just use a normal database index (like a B-tree) to search through embeddings? Explain the core difference between exact lookup and similarity search.

A B-tree index is mainly useful for finding an exact value or values in an ordered range. For example, it can efficiently find a student with a particular roll number.

Embeddings are different because we usually do not want to find an exact vector. Instead, we want to find vectors that are closest or most similar to a given query vector.

The main difference is:

- **Exact lookup:** Finds a specific value, such as roll number 43.
- **Similarity search:** Finds data that is closest in meaning or vector space to the given query.

Since embeddings can have hundreds or thousands of dimensions, a B-tree is not designed to efficiently calculate distances or similarities between them. Vector databases therefore use specialized indexes such as **HNSW** and **IVF** for similarity search.

A4. You're given two vectors: $A = [1, 2, 3]$ $B = [2, 4, 6]$

Calculate the cosine similarity between A and B by hand, showing your steps. What does the result tell you about these two vectors?

Given:

$$A = [1, 2, 3]$$

$$B = [2, 4, 6]$$

The formula for cosine similarity is:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Step 1: Calculate the dot product

$$A \cdot B = (1)(2) + (2)(4) + (3)(6)$$

$$= 2 + 8 + 18 = 28$$

Step 2: Calculate the magnitude of A

$$\|A\| = \sqrt{1^2 + 2^2 + 3^2}$$

$$= \sqrt{1 + 4 + 9} = \sqrt{14}$$

Step 3: Calculate the magnitude of B

$$\begin{aligned}\|B\| &= \sqrt{2^2 + 4^2 + 6^2} \\ &= \sqrt{4 + 16 + 36} = \sqrt{56}\end{aligned}$$

Step 4: Calculate the cosine similarity

$$\cos(\theta) = \frac{28}{\sqrt{14}\sqrt{56}}$$

Since:

$$\sqrt{14}\sqrt{56} = \sqrt{784} = 28$$

Therefore:

$$\boxed{\cos(\theta) = 1}$$

The result is 1, which means the two vectors point in exactly the same direction. In this case, every element of B is twice the corresponding element of A:

$$B = 2A$$

So, they have the same direction but different magnitudes.

A5. Explain why a vector database is essential for RAG specifically. What would break if you removed it from the pipeline?

A vector database is an important part of RAG because it stores document embeddings and helps the system find information that is relevant to the user's question.

When a user asks a question, the question is converted into an embedding. The vector database then compares it with the stored document embeddings and retrieves the most relevant chunks. These chunks are then given to the LLM to generate the answer.

The basic flow is:

User Query → Query Embedding → Retrieve → Augment Prompt → LLM → Answer

If the vector database is removed, the retrieval part of the RAG pipeline would be missing. The LLM would not have an efficient way to find the relevant information from the external documents.

Section C: Advanced RAG

C1. You're building a RAG system for a company's product manuals. These manuals have text instructions along with diagrams and photos showing how to assemble the product. A user asks "how do I attach the side panel," and the answer is shown entirely in a diagram, with no accompanying text.

Explain how you would approach this with multimodal RAG. What would go wrong if you used a text-only RAG pipeline here?

Since the product manual contains both text and images, I would use a **multimodal RAG** approach instead of relying only on text.

The text instructions can be processed and converted into text embeddings. The diagrams and photos can be processed using a vision-language model (VLM) or a multimodal embedding model. When the user asks how to attach the side panel, the system should retrieve the relevant diagram along with any related text.

A simple flow would be:

$$\text{Product Manual} \rightarrow \begin{cases} \text{Text} \rightarrow \text{Text Embedding} \\ \text{Images} \rightarrow \text{Image/Multimodal Embedding} \end{cases}$$

$$\rightarrow \text{Multimodal Retrieval} \rightarrow \text{Relevant Text} + \text{Image} \rightarrow \text{VLM/LLM} \rightarrow \text{Answer}$$

If a text-only RAG pipeline were used, it might retrieve some surrounding text but would not be able to properly understand the information shown only in the diagram. Important details such as arrows, labels, component positions, and the actual assembly steps could therefore be missed.

C2. Briefly compare the two approaches to multimodal retrieval covered in lecture: unified embedding space vs separate indexes with fusion. Which one would you pick for the product manual scenario above, and why?

There are two common approaches to multimodal retrieval.

Unified embedding space: Text and images are converted into compatible vectors and placed in the same embedding space. This allows the system to search both types of information together.

Separate indexes with fusion: Text and images are stored and searched using separate indexes. The results from both searches are then combined before being given to the LLM.

For the product manual scenario, I would use **separate indexes with fusion**. The reason is that text and diagrams can provide different kinds of information. Text retrieval can find the written assembly instructions, while image retrieval can find the relevant diagram or photograph. Their results can then be combined to give the LLM a more complete context.